

Numerical PDEs Homework #0

For all problems asking for a graph: you do not need to write up the method you programmed. Including plots of your solution and convergence is sufficient for full credit. The ideal homework will have one, information-dense and properly labeled figure with a caption and a *brief* discussion. **You must type up your HW.** I don't care if it's word or Tex as long as it's well organized, easy to follow, and submitted as a pdf.

Problem 1: Finite differences Write a matlab code that uses finite differences to compute the first derivative of

$$f(x) = \exp(\sin(x))$$

on the interval $0 \leq x \leq 2\pi$. Use second-order centered differences on internal grid points and first-order one-sided differences at endpoints.

1. Show a graph of the exact analytic solution $f'(x)$ and your numerical solution $f'_n(x)$.

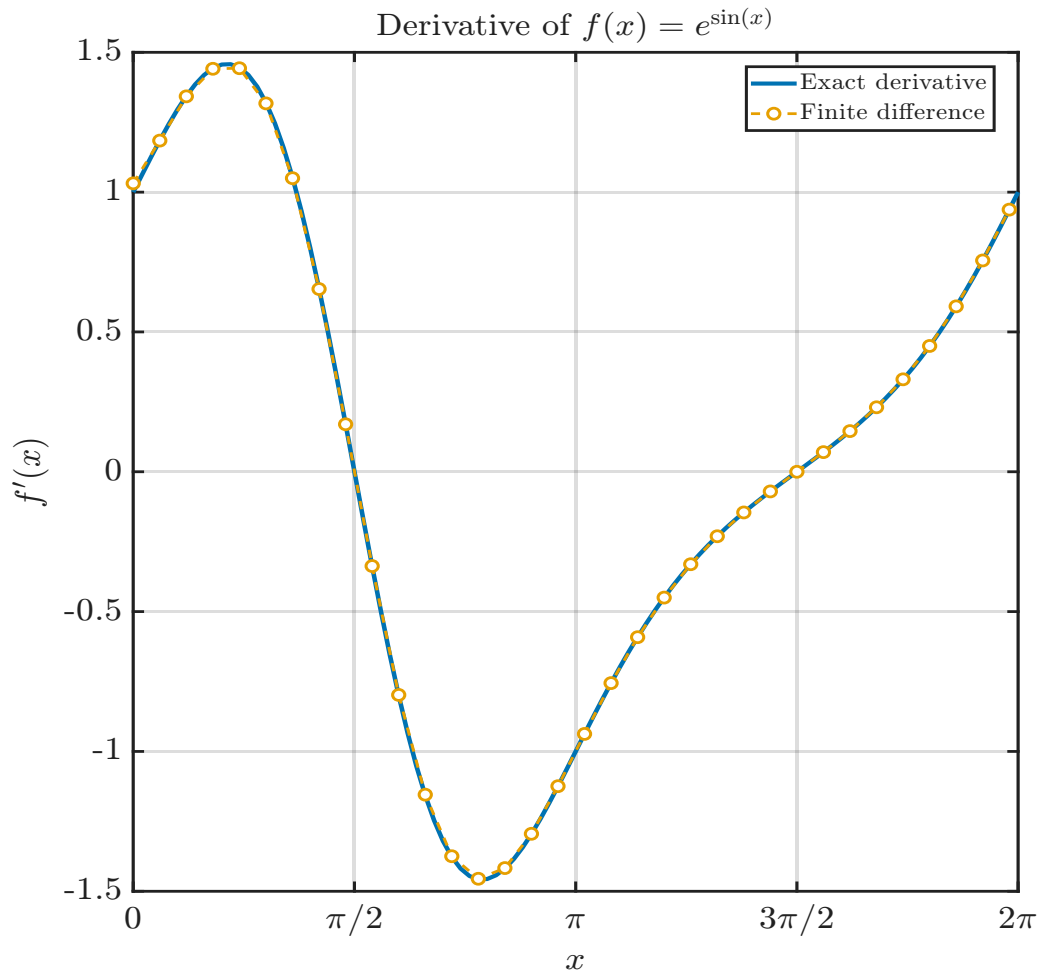


Figure 1: Comparison of the exact derivative of $f(x) = e^{\sin x}$ with the finite difference approximation.

By inspection, the method of approximating the derivative of $f(x)$ by finite differences is remarkably accurate when domain is discretized into grid with $N = 100$ points. To achieve this result, we used first order one-sided differences on the endpoints, left and right-sided on left and right endpoints, respectively, discretized the grid into 100 points, and set the difference between points to $2\pi/100$. We aim to more rigorously characterize the error in the next part.

2. Show a relative error convergence plot by plotting the relative error on the y-axis and the grid size N on the x-axis. Measure relative error as

$$Err = \frac{\|f'(x) - f'_n(x)\|_p}{\|f'(x)\|_p}.$$

Plot the error using both the $p = \infty, 2$ norms in a log-log scale. The relative errors should be lines with slopes 1 and $3/2$ respectively, i.e. it scales with $\frac{1}{N}, \frac{1}{N^{3/2}}$. Extra Credit: The $3/2$ slope is strange, why is that happening?

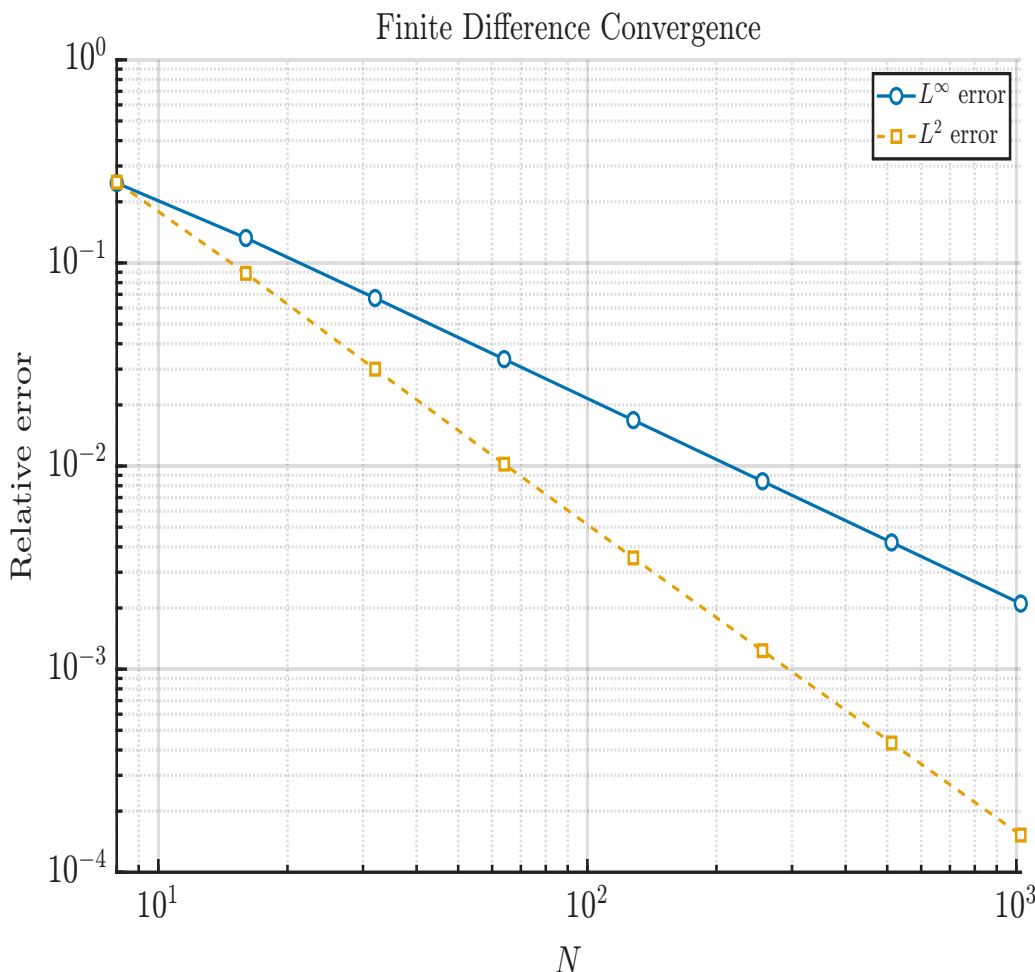


Figure 2: L^∞ and L^2 error log-log plot

As expected, the error decreases with increasing N. In particular, Err scales with $1/N$ with L^∞ norm and $1/N^{3/2}$ with the standard euclidean norm. The L^∞ norm is equivalent

to taking the maximum of the components, so it makes sense that the error scales with $1/N$, as the largest component in this case is the largest error, i.e, the first order error found at the endpoints. Interestingly, we should expect the L^2 norm to grant an error that scales with $1/N^2$, since f_n is a second order approximation. However, it is trivial to show that the error's numerator term is dominated by the endpoints:

$$\begin{aligned} O(\|f'(x)\|_2) &= \sqrt{\sum O(1)^2} = O(\sqrt{N}) \\ O(\|f'(x) - f'_n(x)\|_2) &= \sqrt{2O(1/N)^2 + \sum O(1/N^2)^2} = O(1/N) \end{aligned}$$

from which we can write $O(Err) = O(1/N^{\frac{3}{2}})$. Hence, the first order approximation at the endpoints ultimately determines how the error scales with grid size.

Problem 2: Second-order derivatives Repeat problem 1 but compute the second derivative $f''(x)$. Assume periodicity and wrap around the grid instead of using one-sided finite differences.

For this problem we find the second derivative of the function analytically:

$$f''(x) = e^{\sin x}(\cos^2 x - \sin x)$$

and we use the second order centered approximation:

$$f''(x_i) \sim \frac{f_{i+1} - 2f_i + f_{i-1}}{\Delta x^2}$$

We discretize the grid into 100 points, and take advantage of the periodicity of f to avoid relying on first order one-sided differences at the end points. We utilize the circshift function in matlab to realize this.

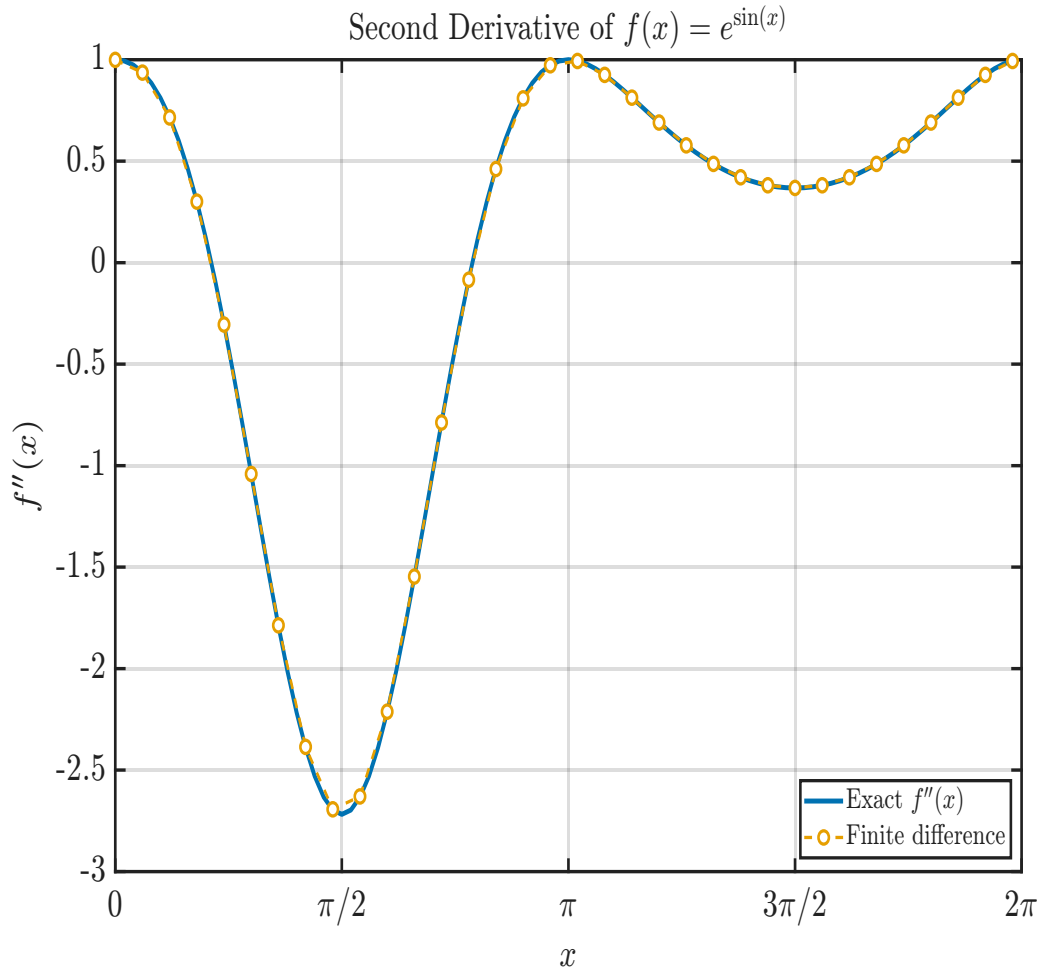


Figure 3: Comparison of the exact second derivative of $f(x) = e^{\sin x}$ with the finite difference approximation.

In this case, we see that the second ordered centered difference method does a good job of approximating the second derivative of f , and we proceed to graph the error under the L^2 and L^∞ norms. Since we are not using first order approximations anywhere, we expect to not see drastic differences in the slopes of the errors in log log plot as in the previous question.

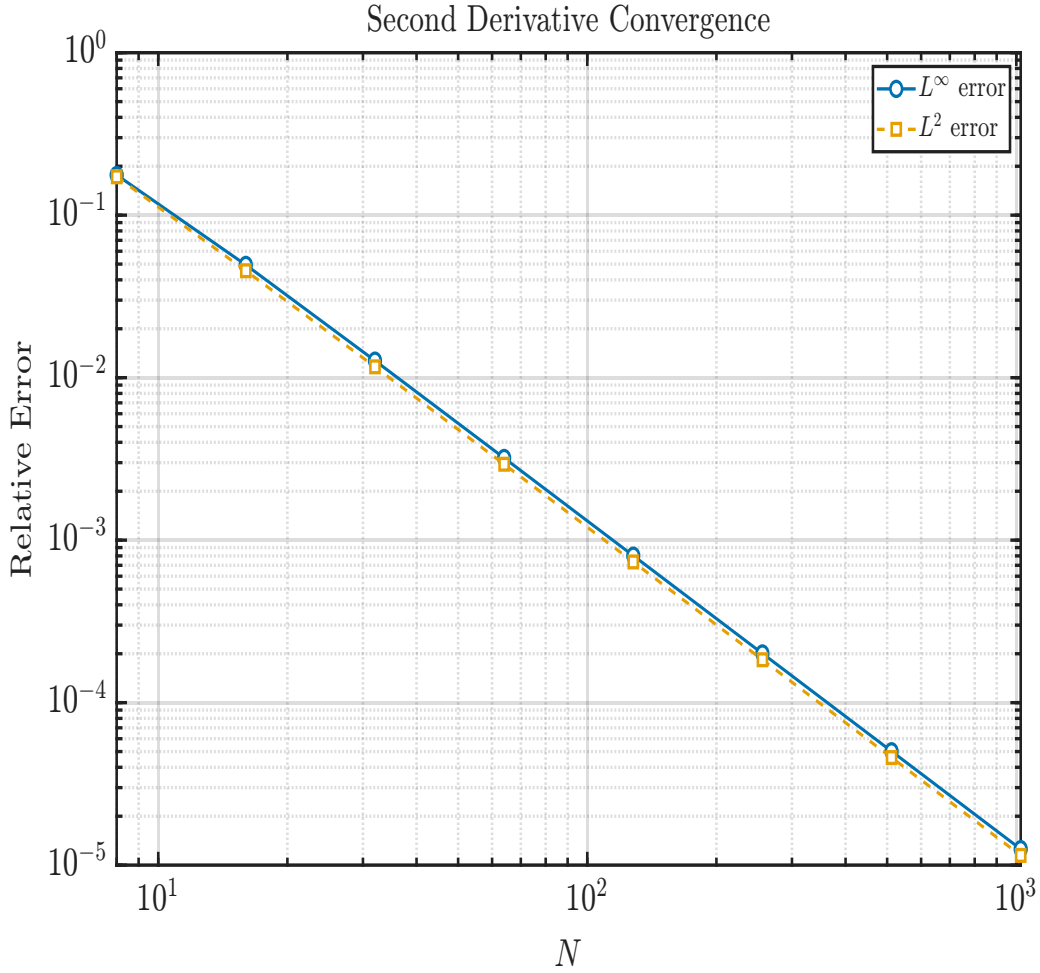


Figure 4: L^2 and L^∞ error log-log plots for approximate solution of second derivative

As expected, the two lines are effectively parallel. In particular, line has slope -2 , and so the error scales with $1/N^2$. Since we do not have any first order approximations, the maximum of the errors for each N is not dominated by the first order approximations, and neither is the numerator of the L^2 norm. Hence, we have what we expect, error of order Δx^2 , or of the order $1/N^2$

Problem 3: ODE Consider the ODE

$$\frac{d^2u}{dx^2} + \sin(x) \frac{du}{dx} + u(x) = f(x)$$

defined on $0 \leq x \leq 5$. Solve the ODE using second-order finite difference methods with the Dirichlet conditions. Use $u(x) = \sin(x)$ to test your solution by finding boundary conditions and the necessary forcing function $f(x)$. Show an error plot that proves your method converges with second order.

We use centered second order differences to approximate u'' and u' , after discretizing the domain so that $h = 5/N$, $x_i = ih$ for $i = 0, \dots, N$, and letting $U_i \sim u_{x_i}$:

$$u''(x_i) \sim \frac{U_{i+1} - 2U_i + U_{i-1}}{h^2}$$

and

$$u'(x_i) \sim \frac{U_{i+1} - U_{i-1}}{2h}$$

Substituting into the ODE:

$$\begin{aligned} \frac{U_{i+1} - 2U_i + U_{i-1}}{h^2} + \sin x_i \frac{U_{i+1} - U_{i-1}}{2h} + U_i &= f(x_i) \\ \left(\frac{1}{h^2} - \frac{\sin x_i}{2h}\right)U_{i-1} + \left(1 - \frac{2}{h^2}\right)U_i + \left(\frac{1}{h^2} + \frac{\sin x_i}{2h}\right)U_{i+1} &= f(x_i) \end{aligned}$$

Letting a_i, b_i, c_i be the coefficients for U_{i-1}, U_i, U_{i+1} , respectively, implies the following relation:

$$a_i U_{i-1} + b_i U_i + c_i U_{i+1} = f(x_i)$$

for $i = 1, \dots, N-1$, which implies the following system:

$$\begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{N-2} \\ 0 & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_{N-1} \end{bmatrix} = \begin{bmatrix} f_1 - a_1 u(0) \\ f_2 \\ f_3 \\ \vdots \\ f_{N-1} - c_{N-1} u(5) \end{bmatrix}.$$

The solution to this system gives us an approximate solution to the ODE with second order error.

We now impose the dirichlet conditions. Assuming, $u(x) = \sin x$ and substituting into the ODE,

$$-\sin x + \sin x \cos x + \sin x = f(x)$$

which implies $f(x) = \sin x \cos x$. We also have $U(0) = \sin 0 = 0$, and $U(5) = \sin 5$. So we get:

$$\begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{N-2} \\ 0 & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_{N-1} \end{bmatrix} = \begin{bmatrix} \sin x_1 \cos x_1 \\ \sin x_2 \cos x_2 \\ \sin x_3 \cos x_3 \\ \vdots \\ \sin x_{N-1} \cos x_{N-1} - c_{N-1} \sin 5 \end{bmatrix}.$$

Solving the system numerically:

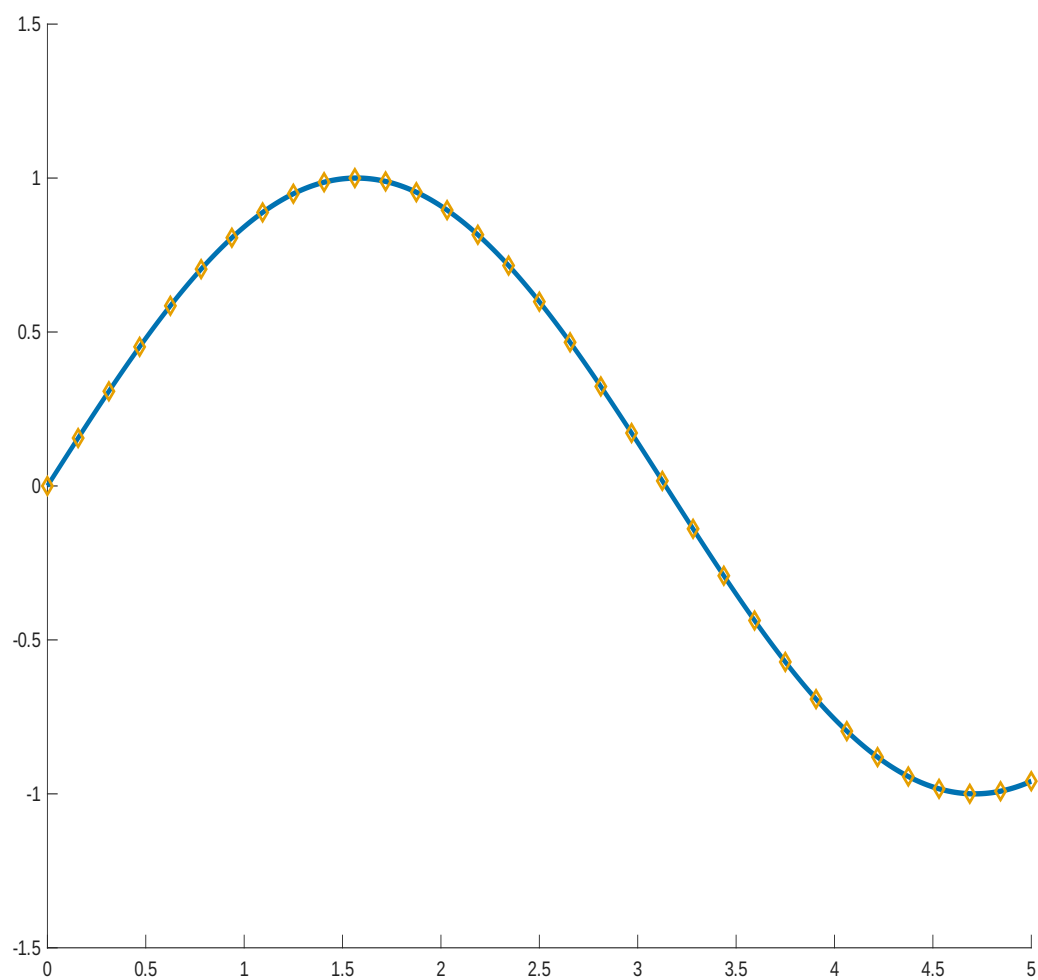


Figure 5: Exact vs Finite Difference Solution to Problem 3

The approximate solution matches the exact solution. We now plot the L^∞ error:

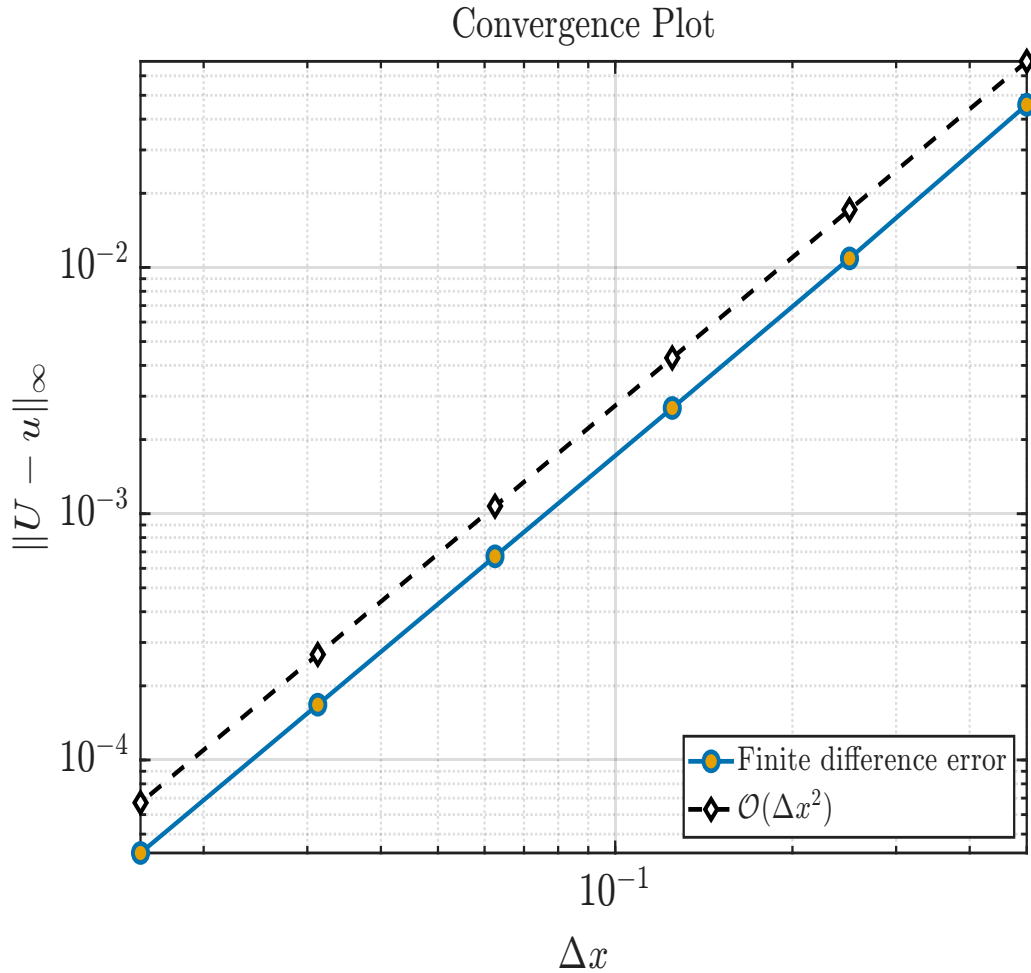


Figure 6: Finite difference error plotted alongside $\mathcal{O}(\Delta x^2)$ with log-log scale

After computing the L^∞ error of our approximate solution to the ODE, we see that as N grows large, Δx approaches 0, and the error scales with $1/N^2$, or Δx^2 . It is parallel to a line of order Δ^2 . This proves the method converges with second order.

Problem 4: Poisson Equation Use finite difference methods to solve the Poisson equation $\nabla^2 u = f(x, y)$ in the domain $x \in [0, 5], y \in [0, 5]$. Use the test solution $u(x, y) = \sin(x) \cos(y)$.

1. Apply Dirichlet conditions on all boundaries so that $u(x, y)$ satisfies the PDE and the boundary conditions, and show a plot of your solution and a second order convergence plot (similar to the first three problems).
2. Modify your code to apply Neumann conditions on all boundaries. **This won't work.** Why not? – both in terms of the linear algebra and in terms of the actual PDE we want to solve. Suggest a solution to the issue.

We begin by finding the forcing function $f(x, y)$ given $u = \sin x \cos y$. Differentiating twice with respect to x, y yields:

$$\begin{aligned}u_{xx} &= -\sin x \cos y \\u_{yy} &= -\sin x \cos y\end{aligned}$$

And so we get that $\nabla^2 u = f(x, y) = -2 \sin x \cos y$. We can also determine the dirichlet boundary conditions:

$$\begin{aligned}u(0, y) &= \sin 0 \cos y = 0 \\u(5, y) &= \sin 5 \cos y \quad u(x, 0) = \sin x \\u(x, 5) &= \sin x \cos 5\end{aligned}$$

We proceed to discretize the domain into $(N + 1)^2$ using a type 1 grid, so $x_i = ih$, and $y_j = jh$ where we have $h = 5/N$, and $i, j = 0, \dots, N$. Using the second-order center difference approximation for both directions, we get the five-point laplacian:

$$\nabla u^2 \sim \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} = -2 \sin x_i \cos y_i$$

We want to solve the linear system $Lu = f$, where $L = I \otimes D + D \otimes I$, where D is simply $D = \frac{1}{h^2} \text{tridiag}(1, -2, 1)$. Solving in Matlab generates the following plot:

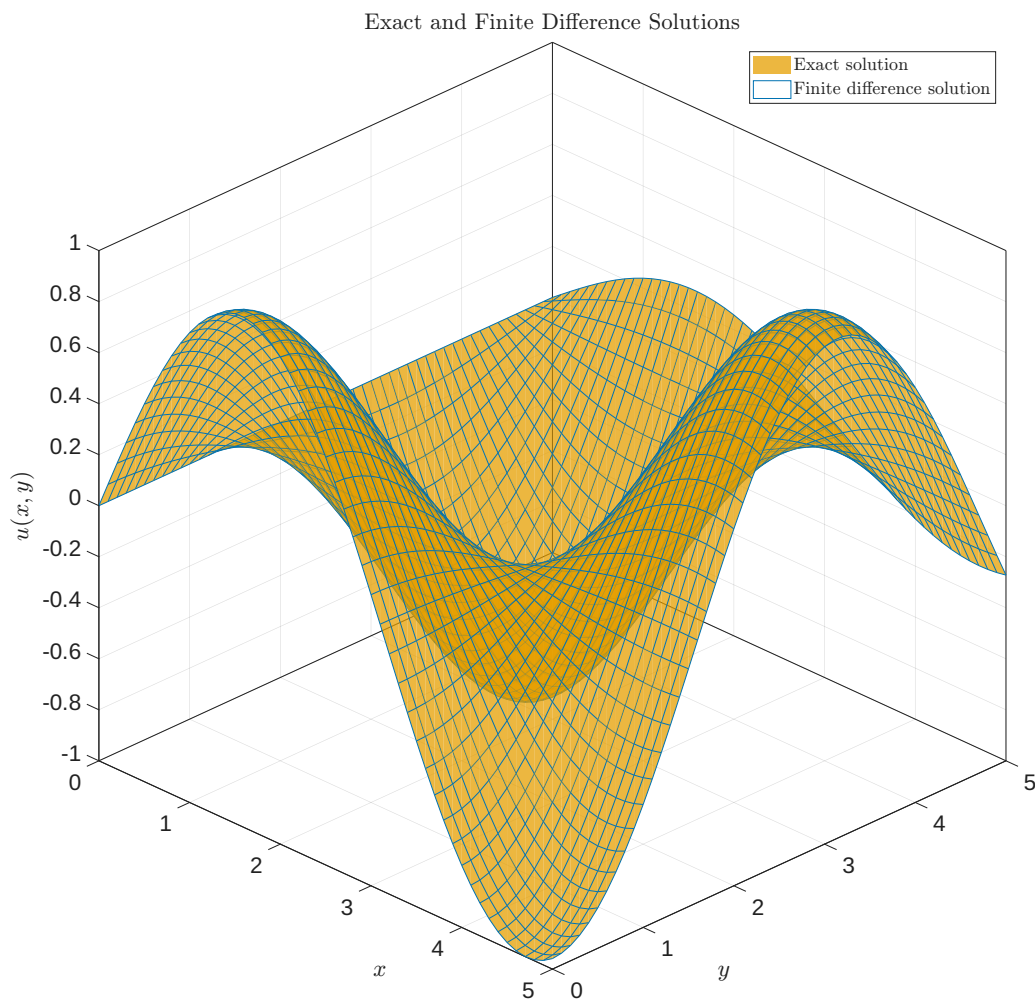


Figure 7: Exact and finite difference solutions to the Poisson equation.

From inspection, the approximate solution shows remarkable similarity to the actual solution. We now rigorously characterize the error by producing a log log plot of it:

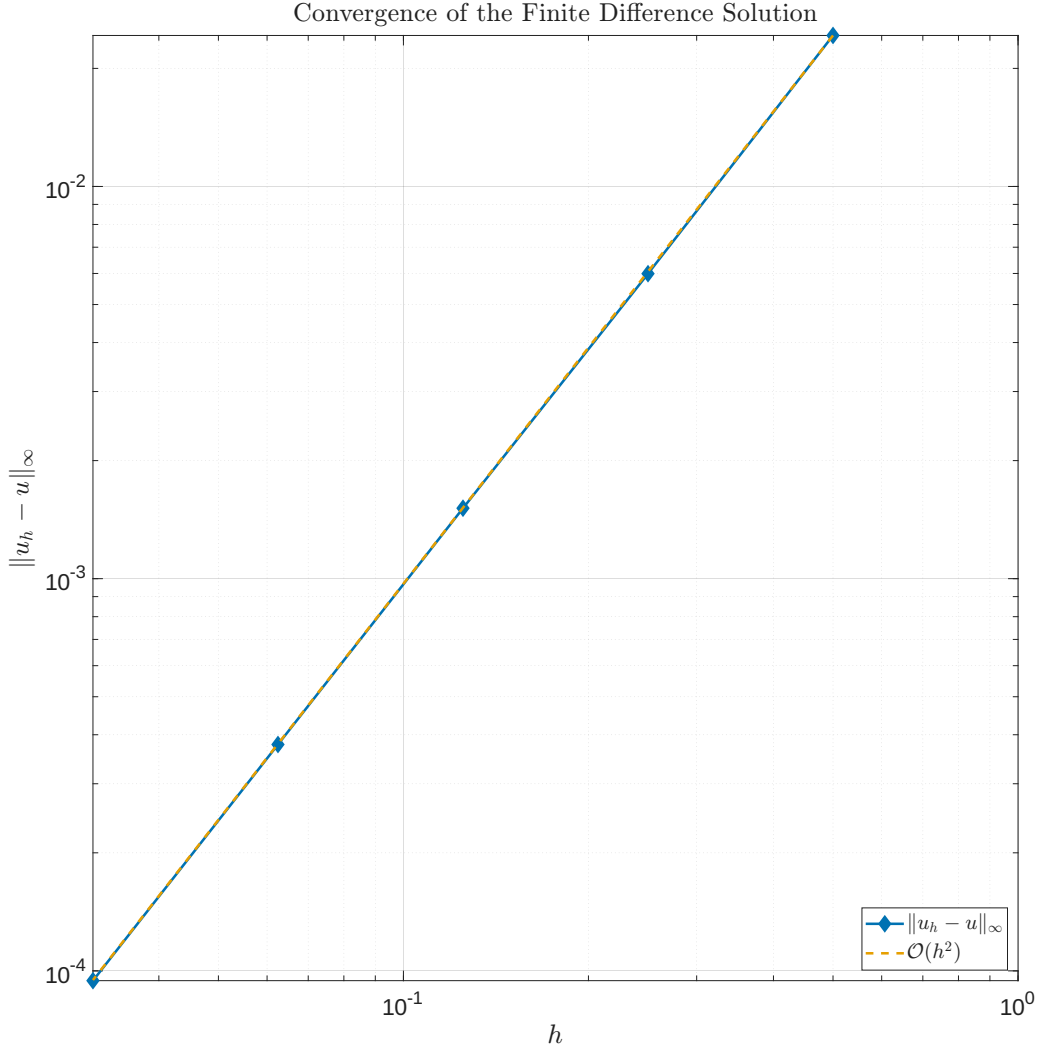


Figure 8: Finite difference solution error plot comparison with $O(h^2)$ error

As expected, the error for the problem scales with $O(h^2)$, in other words, with $1/N^2$.

If we modify the code to apply Neumann conditions on all boundaries, the Poisson equation does not have a unique solution. Simply adding any given solution by any constant is also a solution to the Poisson equation. This has a linear algebra consequence too if one considers the system we originally set out to solve, $Lu = f$. One notes that applying a vector where each component is the same constant yields $Lc = 0$, where c is the vector such that $c_i = c$. Therefore, the kernel of L is non-trivial, which by the invertible matrix theorem implies L is not invertible. Therefore, matlab cannot uniquely solve for u . One possible solution is to additionally state that $u(0,0) = 0$. Then, we get that u must be unique.

AI Statement An LLM (ChatGPT 5.6) was prompted with the textbook on the numberFactory github and the matlab code for each assignment to generate all plot code in the convention required (i.e to implement accessibility features). Further, an OpenCode agent using GLM 5.3 Flash was used to generate the README.md. The matlab code and latex document outside of plots was not generated using LLM.